



East West University

Assignment

Course Code:CSE 360

Course Title:Computer Architecture

Semester: Spring 2025

Section:1

Course Instructor

Dr. Md. Nawab Yousuf Ali

**Professor, Department of Computer Science and Engineering,
East West University.**

Submitted by

Name:Nuran Farhana Prova

ID:2023-1-60-075

Department of CSE

Date:23 May 2025

Game:Penguin Math Race

Abstract: Penguin Jump Multiplication Race is a C-programmed educational game in which a human competes against a computer to exercise multiplication skills during a turn-based race. By using registers and memory management, the game shows how control goes from register to register and influences data. Correct answers to random multiplication questions help players advance step by step along a single track. The game has a register structure that copies how basic processors store information. As the game continues, students answer math problems; getting the right answer helps them proceed and earn points. With this scheme, each answer the computer gives is random, so players cannot predict what to expect. Important features are saving and loading game states with file I/O, displaying a race track and clearly explaining main programming concepts with separate modules. When a player travels all the way to the far side of the track, the game is over and a message tells you the result. Penguin Jump is intended for learning by allowing students to implement computer architecture using a fun programming game.

Introduction: Penguin Jump Multiplication Race teaches both multiplication and various computer architecture terms on the C console. Both the human and the computer players move on a straight track, answering multiplication questions in succession. If players give correct answers, they move on, playing a race together. The project follows some important architectural methods such as register simulation, memory management, control flow and file I/O. You get to enjoy learning how data is handled in a simple computing setup.

Game Overview:

"Penguin Jump Multiplication Race" is a two-player (human vs. computer) console game implemented in C. Players race to reach the end of a linear track by correctly solving multiplication questions.

Gameplay Flow

Initialization:

Both players start at position 0 with a score of 0. The user chooses to start a new game or load a previously saved one.

Turn-Based Logic:

Players take alternating turns. Each turn generates a random multiplication question (operands 1–10). The human player inputs their answer. The computer responds using a 70% probability of giving the correct answer.

Movement and Scoring:

Correct answers move the player forward by 1 step and award 1 point.

Incorrect answers result in no movement or points.

Game Display:

The track visually displays player positions. Symbols used:

| P | = Human player

| C | = Computer

| B | = Both players in the same cell

Persistence (Save/Load):

Game state is saved to `savegame.dat` after every turn. Players can resume progress by loading the saved game.

Win Condition:

The first player to reach position 10 wins. The winner is announced, and the save file is deleted.

Architectural Principles Applied

1.Register Simulation:The `RegisterSim` struct mimics CPU registers. It holds `operand1`, `operand2`, and `result`, simulating data storage and arithmetic operations as performed in processor registers.

2.Memory Management:Dynamic memory allocation is used (`malloc` and `free`) for player objects. This simulates how programs manage memory in the heap, a key concept in both system programming and architecture.

3.Control Flow and Branching:Game logic uses `if`, `else`, and `while` to represent conditional execution and loop constructs. Alternating turns reflect instruction sequencing and branching in CPU operations.

4.File I/O (Persistent Storage):The game saves and loads player states using binary file operations (`fwrite` and `fread`). This mirrors how data is written to and read from memory.

or disk, demonstrating non-volatile storage behavior.

5.Random Input Simulation:The `rand()` function introduces variability, simulating dynamic input data.The computer's probabilistic answer selection (70% correctness) adds uncertainty, similar to real-world sensor or user input.

6.Modular Design:The program is broken into multiple functions (e.g., `initPlayers`, `playTurn`, `saveGame`), promoting modularity.This reflects how systems use subroutines or modules to manage complexity.

7.State Machine Behavior:The game flow operates like a finite state machine.Each turn represents a state, and transitions depend on player input and correctness of answers.

8.Data Representation:Player data and game state are structured using structs, representing how complex data types are organized and stored in memory.

Challenges Faced and Solutions

Challenge 1: Managing Player State Across Turns

Issue: Maintaining updated player positions and scores across alternating turns.

Solution: Used structured data types (`struct Player`) and saved game state to a file after each turn using binary file I/O.

Challenge 2: Implementing Realistic Computer Player Behavior

Issue: Making the computer player act in a non-deterministic yet fair manner.

Solution: Introduced a probability model (70% chance of correct answers) and random deviations to simulate intelligent but imperfect behavior.

Challenge 3: Ensuring Code Modularity

Issue: Keeping the code readable and maintainable as functionality increased.

Solution: Divided game logic into separate functions such as `initPlayers()`, `playTurn()`, `saveGame()`, and `displayTrack()`.

Challenge 4: Cross-Platform Screen Clearing

Issue: Clearing the screen worked differently on Windows and Linux.

Solution: Used preprocessor directives (`#ifdef _WIN32`) to conditionally use `system("cls")` or `system("clear")`.

Insights Gained About Computer Architecture Through Coding

- **Understanding Register Operations:**

Simulating a register (RegisterSim) reinforced how data is loaded, operated on, and stored in processor registers during instruction execution.

- **Appreciating Control Flow:**

The logic structure of the game closely mimicked how control units in CPUs manage decision-making and execution flow.

- **Memory Layout and Management:**

Using dynamic memory allocation (malloc, free) highlighted the importance of managing heap memory, much like in system-level programming.

- **Role of Persistent Storage:**

Implementing save/load functionality emphasized the relationship between volatile (RAM) and non-volatile (disk) storage in computing systems.

- **Simulating State Machines:**

The turn-based game structure provided a practical example of finite state machines, commonly used in hardware design and low-level software.

Description of Any Low-Level Simulation or Optimizations Performed

- **Register-Level Simulation:**

The game uses a RegisterSim struct to represent arithmetic logic unit (ALU) operations. This simulates how multiplication instructions are executed within CPU registers.

- **Manual Memory Management:**

Memory for players is manually allocated and freed, simulating how low-level systems manage memory without automatic garbage collection.

- **Binary File Handling:**

Saving and loading the game state in binary format simulates raw memory operations, closer to how data is stored and retrieved in actual hardware.

- **Conditional Compilation:**

The use of `#ifdef` for cross-platform compatibility reflects system-level optimizations done at compile time.

Suggestions for Future Improvements

Enhanced User Interface:

Add color and ASCII animations for a more engaging experience.

Difficulty Levels:

Introduce adjustable difficulty with more complex arithmetic (e.g., division, larger numbers).

Multiplayer Mode:

Allow two human players to compete on the same or different machines using network sockets.

Real Register Emulation:

Extend the register simulation to include flags (e.g., zero, carry) and more operations for closer alignment with actual CPU behavior.

Better Save System:

Allow multiple save files and include timestamps for each saved game.

Output:

[illegible]

